
Data Structures

BS (CS) _Fall_2025

Lab_02 Task



Learning Objectives:

1. Class Templates with unit testing (Gtest)

Class Templates in C++

Task 0: Simple Generic Holder

Create a template class `Holder<T>` that stores a single item of any type.

Data Members:

- **T item** : the stored value.

Functions:

- **Constructor**: initialize the item.
- **setItem(T value)**: set the stored item.
- **getItem()**: return the stored item.
- **displayType()**: prints the data type of the stored item using `typeid(item).name()` from `typeinfo` header.

Test Cases:

- Create `Holder<int>`, `Holder<double>`, and `Holder<string>`.
- Set and get values for each.
- Print stored type and value.

Task 1: Generic Seating Arrangement

You are required to design a class template `Seating<T>` that manages a 2D seating arrangement (like an exam hall or a bus).

The `Seating<T>` class should represent seats arranged in rows and columns.

- Each seat can hold a value of type `T`.
- `T` could be a student name (`string`), a student ID (`int`), or simply an occupied/free status (`bool`).
- The class should provide functions to assign a seat, vacate a seat, check vacancy, and print the seating chart.

Additionally, you will implement a specialized version of this class for `bool`.

- In this case, each seat only stores whether it is occupied (`true`) or vacant (`false`).
- The `print()` function should show `O` for occupied seats and `-` for vacant seats.

Member Functions:

Constructor

- Takes number of rows and columns, allocates a 2D array of type T.

assignSeat(row, col, value)

- Assign a value of type T to a seat (e.g., "Ali", 101, or true).

vacateSeat(row, col)

- Empties a seat (removes value or sets false for bool).

isVacant(row, col)

- Returns whether a seat is empty.

printSeating()

- Prints the seating chart in a clean grid format.
- For bool specialization: show O for occupied and - for empty.

Test Case Descriptions for Seating<T>

- **Assign and Retrieve String Seat**
 - Assign "Ali" to (0,0) in Seating<string>.
 - Verify that the seat contains "Ali".
- **Vacate String Seat**
 - Assign "Sara" to (0,1), then vacate it.
 - Verify that (0,1) is vacant afterwards.
- **Assign and Retrieve Integer Seat**
 - Assign 101 to (1,0) in Seating<int>.
 - Verify that (1,0) contains 101.
- **Boolean Specialization , Occupied/Vacant**
 - In Seating<bool>, assign (0,0)=true and (0,1)=false.
 - Verify that (0,0) is occupied and (0,1) is vacant.

Task 2:

You have to make a class for an online Banking system. The bank keeps a record of all the transactions made along with the current balance. The bank allows a specific number of transactions to be made.

You have to define a template class **OnlineBank** which can store different types of balance e.g. int, double, float etc. Bank will have same type of elements at a time. The **OnlineBank** class should have the following member variables:

- **T currentBalance** the current balance in the bank.
- **T* withdrawals** A pointers to set the withdrawals made from the account. Before making a withdrawal, it should be checked if it's smaller than or equal to the current balance.
- **T* deposits** A pointers to set the deposits made from the account.
- **numOfTransactions** this is the number of withdrawals and deposits that can be made
- **counterWithdrawal** this is the number of current withdrawals made
- **counterDeposits** this is the number of current deposits made

The class should consist of following functions:

- **Constructor** with current balance and number of transactions as parameter
- **isWithdrawal** checks if the number of current withdrawals is less than the numOfTransactions
- **isDeposit** checks if the number of current deposits is less than the numOfTransactions
- **makeWithdrawal** which will return true if the value passed in the parameter is less than or equal to the current balance and its counter is less than the limit. It will deduct the given amount from current balance and update it.
- **makeDeposit** which will get a value to be added in the current balance if its counter is less than the limit
- **getCurrentBalance()** will return the current balance
- **print** this will print the current balance

Test Cases:

Create a comprehensive test class for the **OnlineBank** template class. Create separate instances for **int**, **float**, **double** data types and perform the following test cases:

- Create OnlineBank instances with different initial balances and transaction limits
- Test multiple valid deposits within transaction limits and verify balance updates
- Test multiple valid withdrawals where amount $\leq$ currentBalance and verify balance updates
- Attempt invalid withdrawals where amount $>$ currentBalance and verify rejection
- Test transaction limits by exceeding both deposit and withdrawal limits
- Perform mixed deposits and withdrawals in random combinations
- Test boundary cases like zero balance, minimum amounts, and exact balance withdrawals
- Test data type specific operations (whole numbers for int, decimals for float/double)